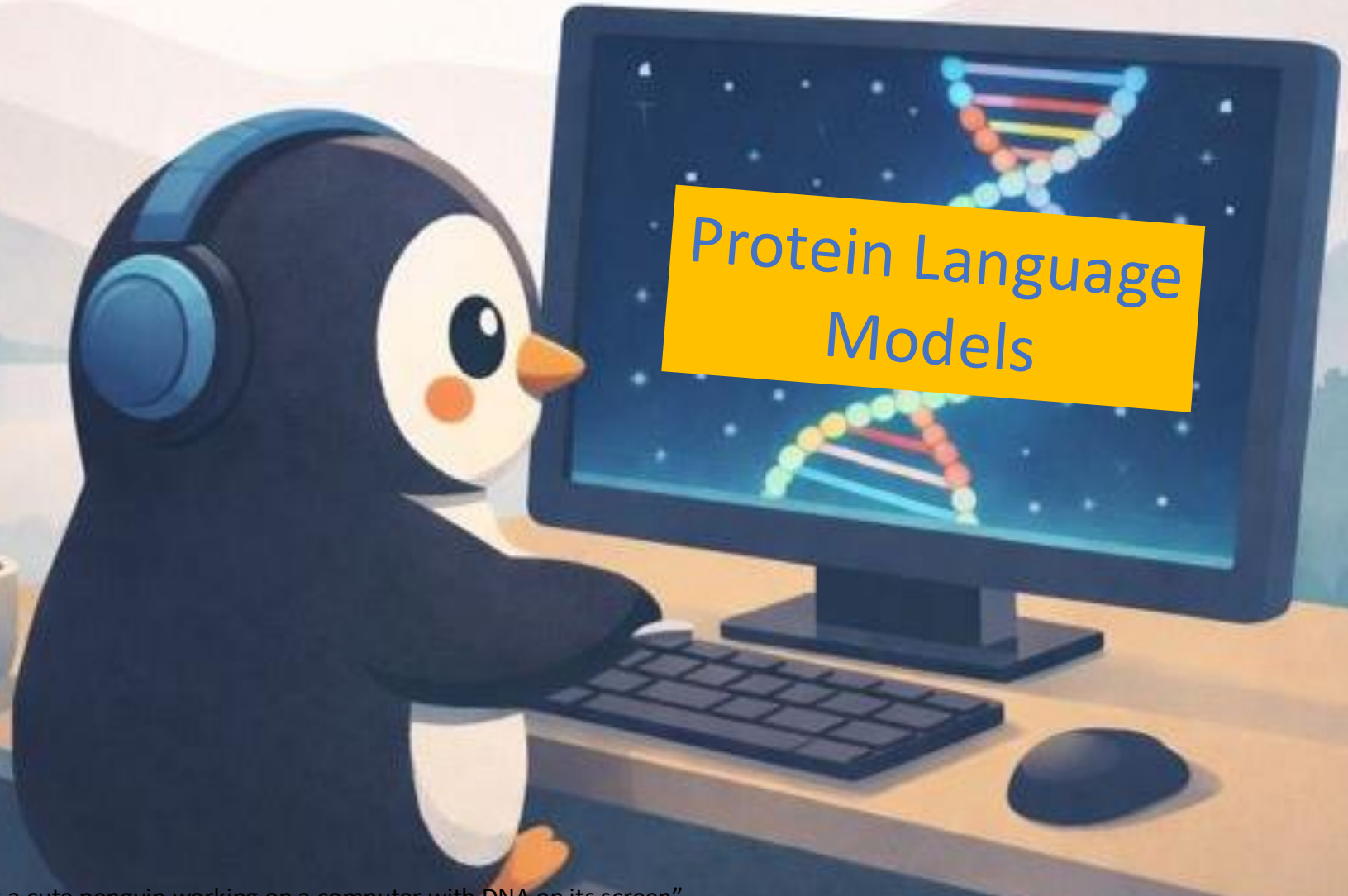


Machine Learning for Biology and Health

CSCI 1851
Spring 2026

Ritambhara Singh

April 23, 2026
Thursday



Today's goal – learn about protein language models and how to use them

(1) Protein Language Models

(2) Hugging Face

(3) LoRA fine-tuning

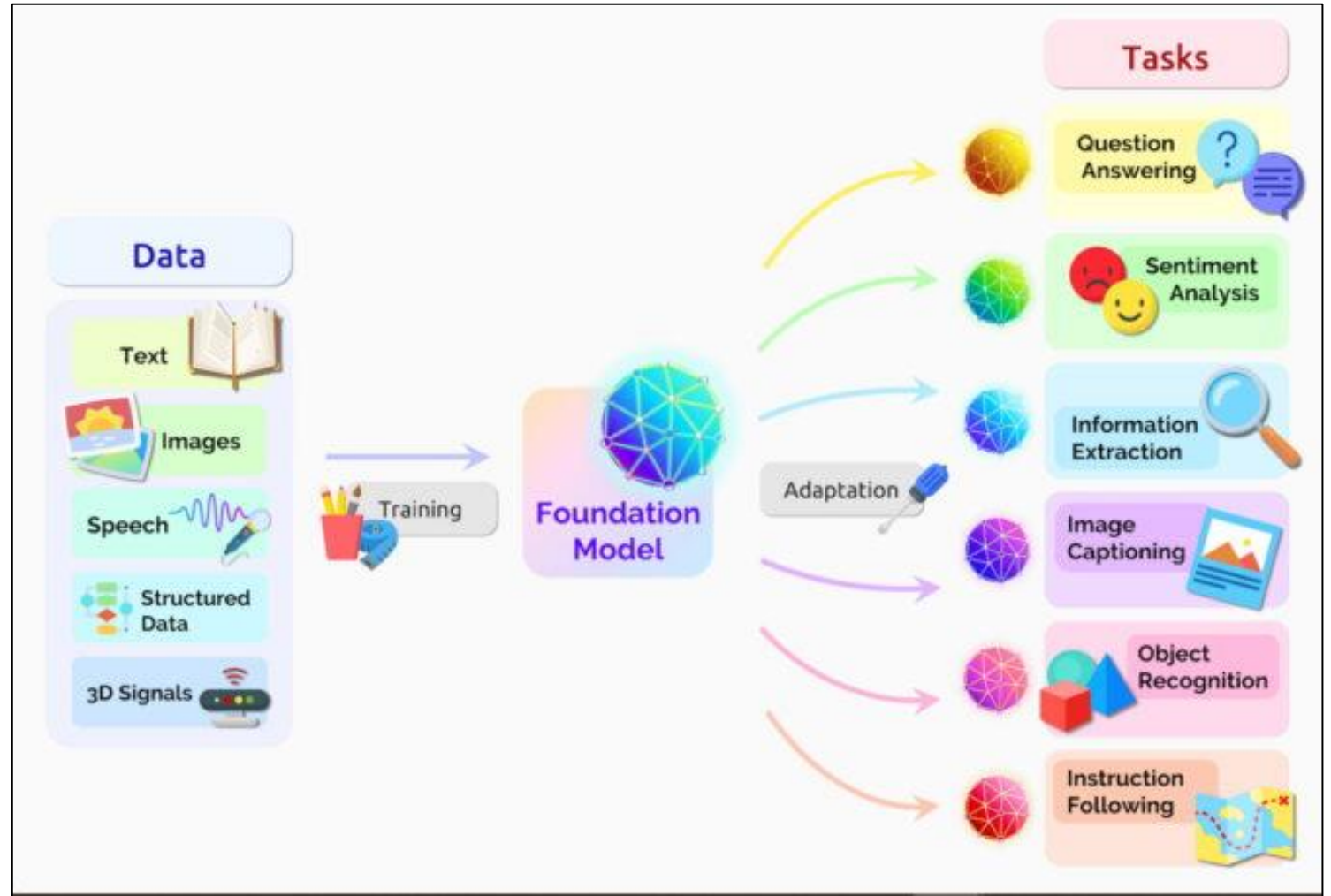
(3) Class activity: ESM model for protein sequences

Foundation Models (FMs) for generalizing across tasks

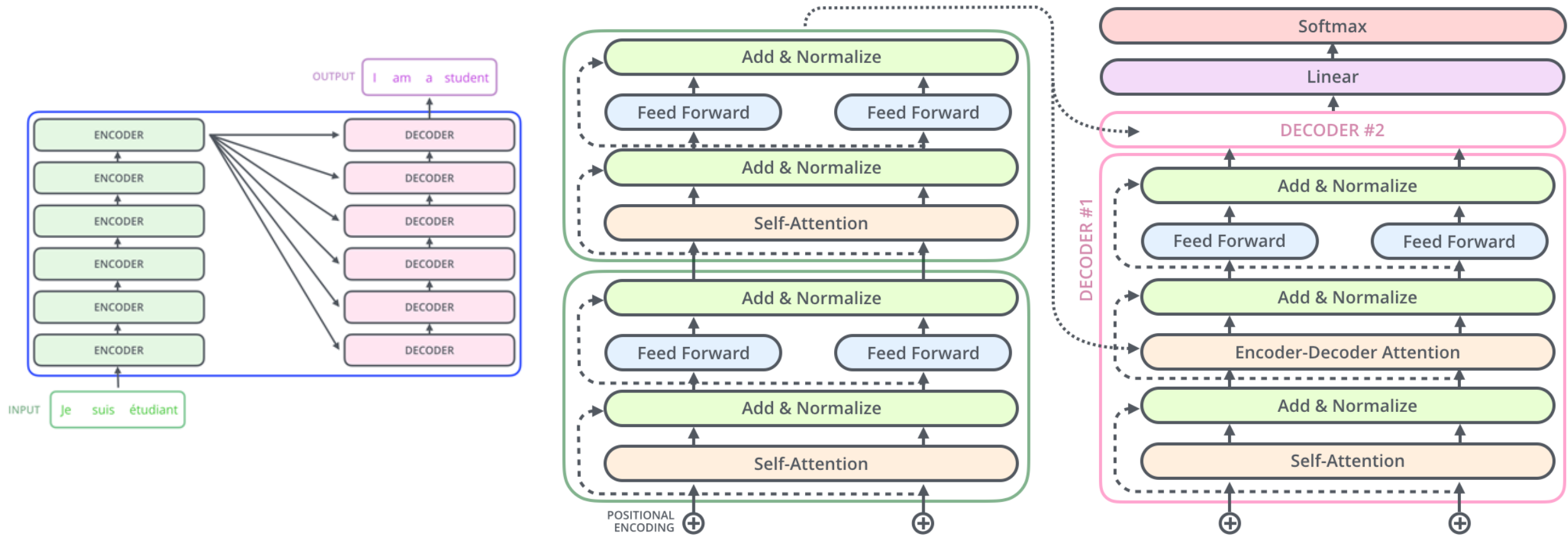
Foundation models are **large** neural networks.

Trained on **large amounts** of **unlabeled** data (self-supervised learning).

Unlike more narrow traditional machine learning models, foundation models can be **easily adapted to many different tasks** without new training data.



Encoders and Decoders Together



Foundation Model Architectures

Encoder-Only



Bidirectional Attention

Sees entire input sequence at once

Training: Masked language modeling (MLM)

Examples: BERT, RoBERTa



Best For:

- Classification tasks
- Named entity recognition
- Question answering
- Embeddings & similarity

Decoder-Only



Causal/Autoregressive

Only sees previous tokens (left-to-right)

Training: Next-token prediction

Examples: GPT, Claude, LLaMA



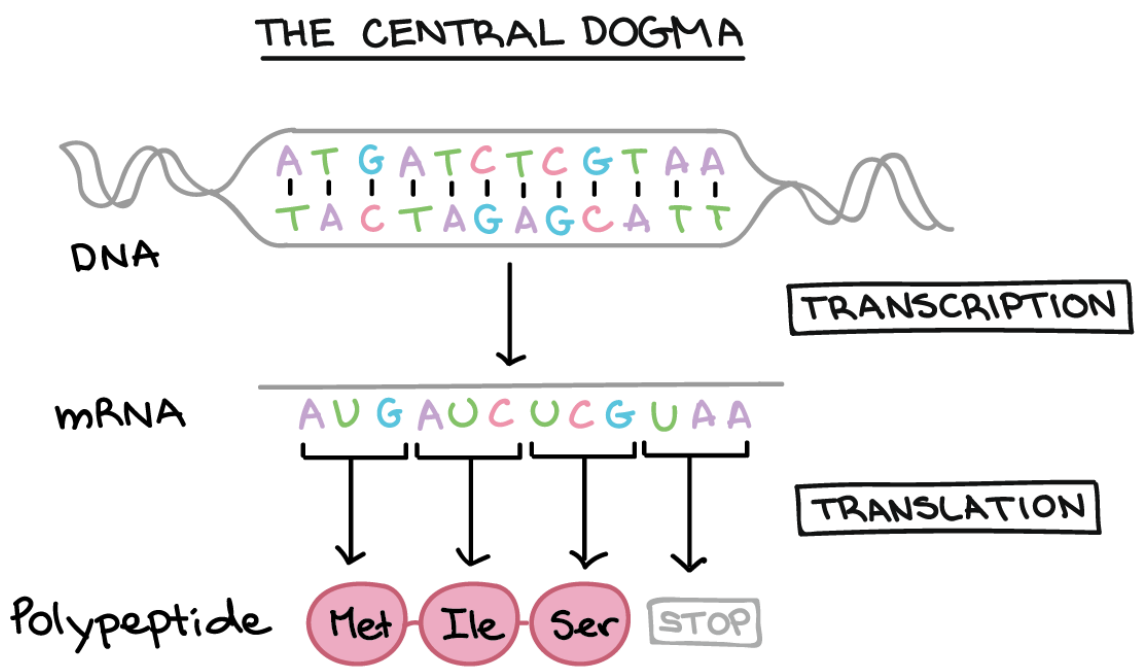
Best For:

- Text generation
- Instruction following
- Dialogue & conversation
- Code generation

Note: Decoder-only models have become dominant for large foundation models due to better scalability

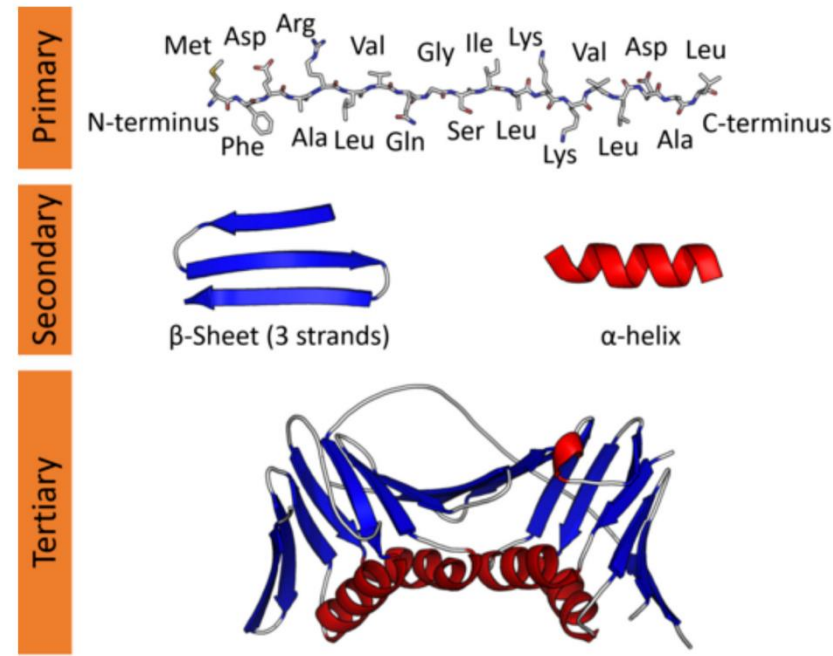
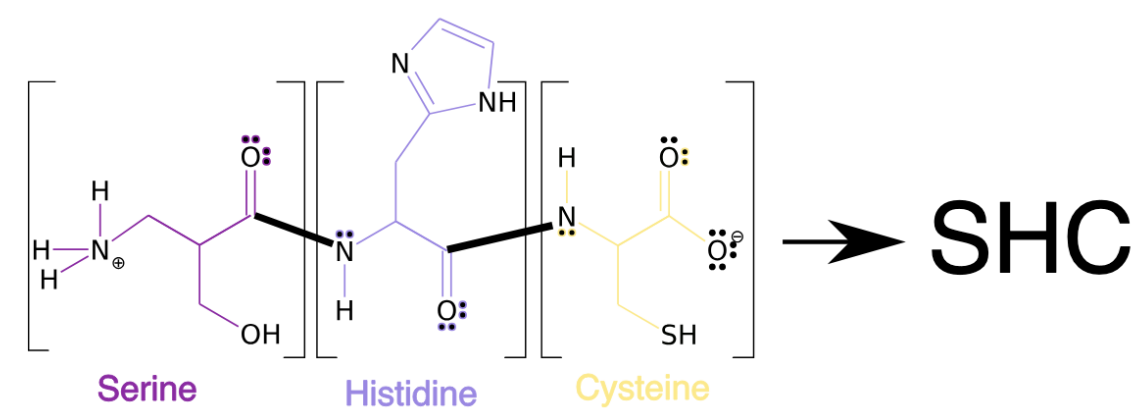
Protein Language Models

Proteins



A protein sequence is made up of amino acids.

There are 20 amino acids



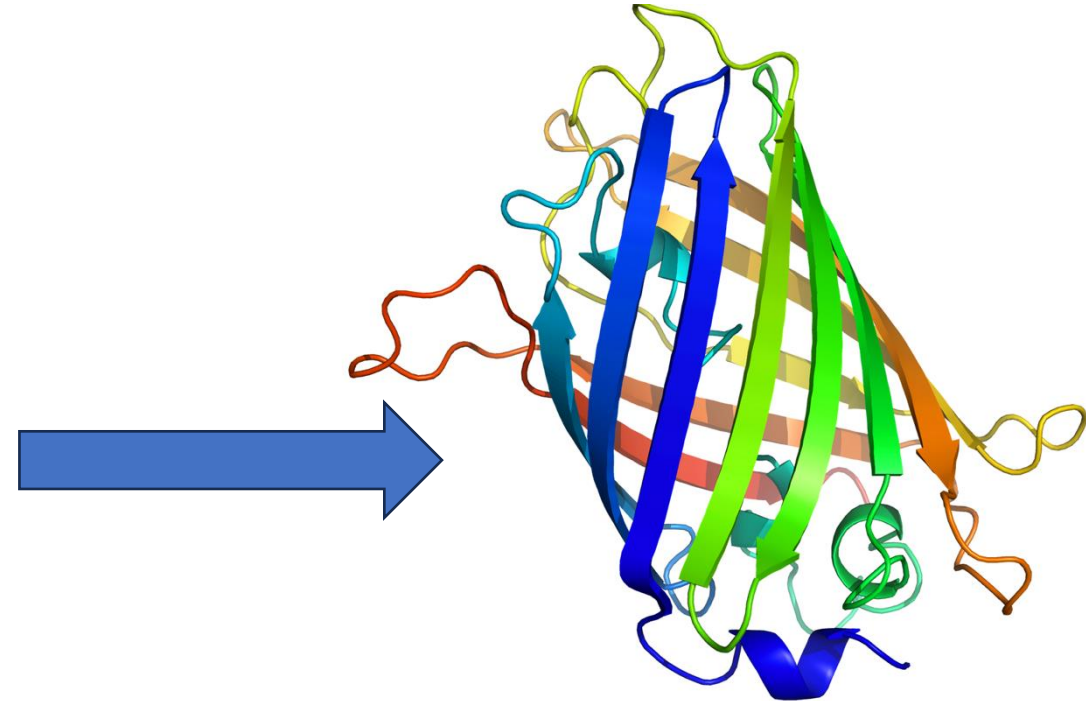
Protein prediction tasks

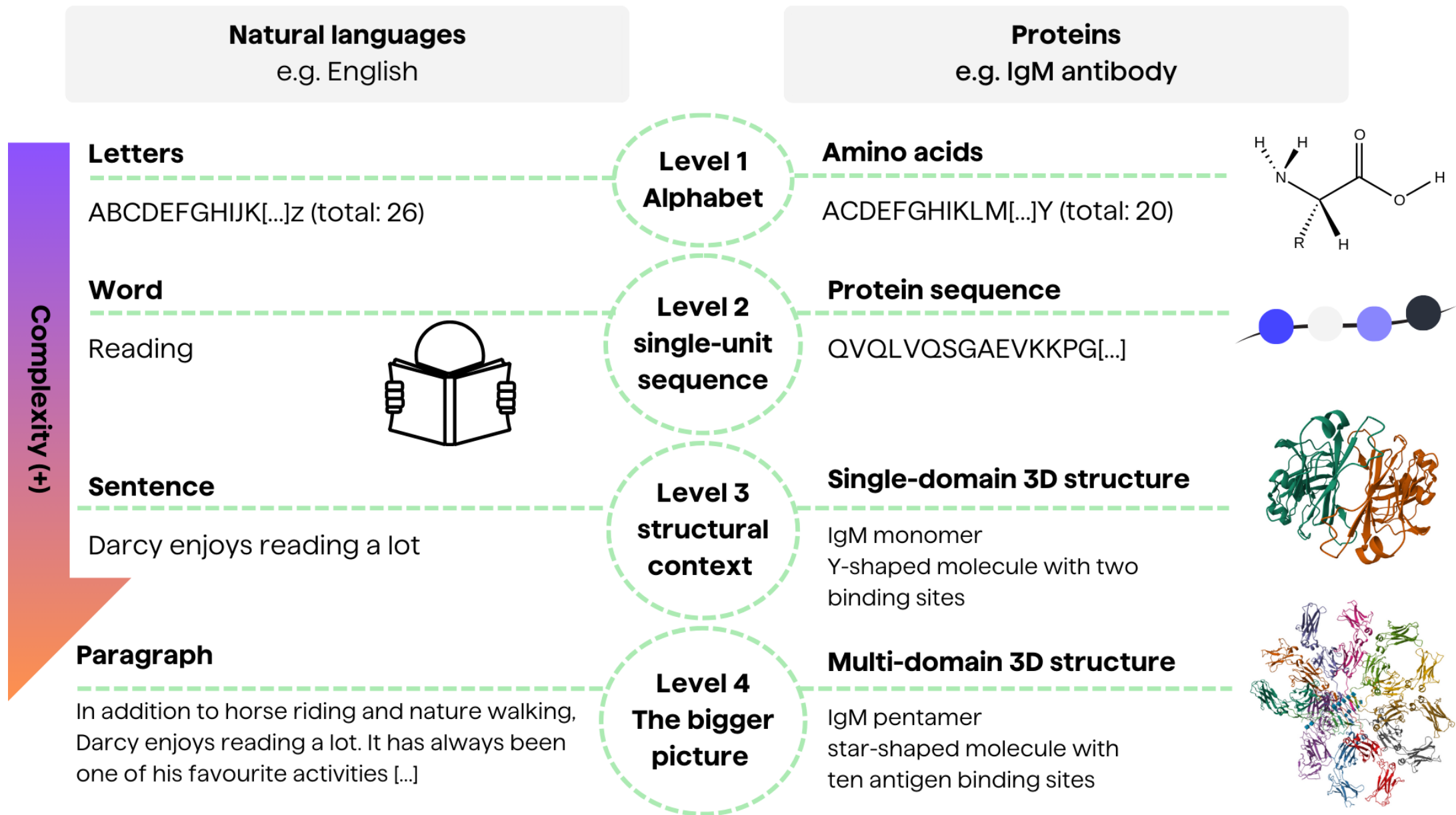
What type of learning?

What type of prediction task?

MSKGEELFTG	VVPILVELDG	DVNGHKFSVS	GELEGDATYG	KLTLKFICTT
GKLPVPWPTL	VTFESYGVQC	FSRYPDHMKQ	HDFFKSAMPE	GYVQERTIFF
KDDGNYKTRA	EVKFEGDTLV	NRIELKGIDF	KEDGNILGHK	LEYNYNSHNV
YIMADKQKNG	IKVNFKIRHN	IEDGSVQLAD	HYQQNTPIGD	GPVLLPDNHY
LSTQSALSKD	PNEKRDHML	LEFVTAAGIT	HGMDELYK	

Any famous model comes to mind?





Protein sequence datasets (>100 million sequences)

```
A0A125NTU3_HYPSL/7-228      .....ELFTGVVPILVELDGDVNGHKFSVSGEGEGDATYGKLTCLKFICTTGK-LPVPWPPTLVTTFGYGLQCFARYPD
A7S3R2_NEMVE/58-276         .....SLSKDAMKLFHMEGVSNGHCFEIQGVGEGKAFDGEHWSKLCVVKGKHLPFSDILMPSMSYGTKQFAKYP
C3ZLQ0_BRAFL/370-580        ..msvptn-----LDLHIYGSINGMEFDMVGGGSGNPNDGSLSVNVKSTKGA-LRVSPLLVGPGLGYGHYQYLPFPD
A7S1H3_NEMVE/1-82           .....-----MFYGSKAFKAYPD
A7RGQ5_NEMVE/1-174          .....-----MKLHFKLEGSVNGHCFEIQGVGEGKPFEGEQWAKHCVVKGKHLPFSLDIIMPNI-----TFAYKYPD
C3YKP7_BRAFL/5-213          ....athe-----IHLHGSVNGHEFDLVGSGKGDPAKSLVTEVKSTMGP-LKFSPLMLIPHLGYGYQYLPYPD
U6M5D0_EIMMA/34-255         .....ELFTGVVPILVELDGDVNGHKFSVSGEGEGDATYGKDLCKFICTTGK-LPVPWPPTLVTTFGYGLMCFARYPD
C3YR99_BRAFL/49-259         ...ptthe-----LHIFGSFNGVEFDMVGRGIGNPNPDGYEELNLKSTKGA-LKFSWPILVPOIGYGFHQYLPYPD
W2JZJ6_PHYPR/96-317         .....ELFTGIVPILIELNGDVNGHKFSVSGEGEGDATYGKLTCLKFICTTGK-LPVPWPPTLVTTLSYGVQCFSRYPD
C3YRA0_BRAFL/13-103         eplptthe-----LHIFGSFNGVEFDMVGRGEGNPKDGSQNLHLKSTKGA-LQFSPWMLVPHIGYGFYQYLPYPD
U6GSR1_EIMAC/7-228          .....ELFTGVVPILVELDGDVNGHKFSVSGEGEGDATYGKLTCLKFICTTGK-LPVPWPPTLVTTFGYGLMCFARYPD
C3YK10_BRAFL/3-212          ..nksvpt-----NLDLHIYGSINGMEFDMVGGGSGNPNDGSLAVNVKSTKGA-LRVSPLLVGPGLGYGHYQYLPFPD
C3YRA2_BRAFL/11-221         ...pkthe-----LHIFGSFNGVEFDMVGRGIGNPNNEGSEELNAKFTKGP-LKFSPIYILVPHLGYAYYQYLPFPD
C3YKQ3_BRAFL/5-214          ....athd-----IHLHGSINGHEFDLVGSGKGDPAKSLVTTAKSTKGA-LKFSPLYLMIPHLGYGYQYLPYPD
A0A2B4RJ70_STYPI/16-192     .....e-IIQDDMKMEYEMKGWVNGHEFTIEGEGNGKPYEGKQTANFKVITGAPLSFSFDIPSSVFGYGNRCFTRYPE
C3YRA1_BRAFL/52-263         ...pkthe-----LHIFGSFNGVKFDMVGEETGNPNNEGSEELKLKSTNGP-LKFSPIYILVPHLGYAFNQYLPFPD
C3YKP9_BRAFL/5-212          ....tahd-----LHIFGSVNGAEFDLVGGGKGNPNPDGTLETSVKSTRGA-LPCSPLLIGPNLGYGFYQYLPFPG
A0A1V4T0E4_9GAMM/10-189    .....ELFTGVVPILVELDGDVNGHKFSVSGEGEGDATYGKLTCLKFICTTGK-LPVPWPPTLVTTFTYGVQCFSRYPD
C3YRA3_BRAFL/5-214          ....tthe-----VHVYGSINGVEFDLVGSGKGNPKDGSSEIQQVKSTKGP-LGFSPIYIVPNIGYGFHQYLPFPD
C3YKQ1_BRAFL/5-213          ....tthd-----LHIFGSVNGAEFDLVGGGKGNPNPDGTLETSVKSTRGA-LPCSPLLIGPNLGYGFYQYLPFPG
A0A2B4RBW9_STYPI/44-153    .....npy-----
U6M9S0_EIMMA/1-78          .....q-----
J8VIQ3_9SPHN/6-227          .....ELFTGVVPILVELDGDVNGHKFSVSGEGEGDATYGKLTCLKFICTTGK-LPVPWPPTLVTTFGYGLQCFARYPD
A0A2B4R4L6_STYPI/1-109     .....m-----KYPA
C3YKQ0_BRAFL/6-195          ....ahdc-----HMFSGSINGHEFDLVGGGNGNPNDGTLETKVRSTKGA-LPFSPIVILAPNLGYGYHQYLPFPA
A7S1H5_NEMVE/2-221         .....SLSKDAMKHLILEGSVNGHCFEIHGEGEGKAFEGEQWSKFTVKKGGPLPFSFDLIAPCLKYGSKPFVKYPD
```

Sequential, discrete structure. Proteins are linear chains of amino acids drawn from a fixed "alphabet" of ~20 standard residues, much like sentences are sequences of tokens from a vocabulary.

Context-dependent meaning. Just as the meaning of a word depends on surrounding words, the functional role of an amino acid depends on its sequence context. A leucine in a hydrophobic core behaves very differently from one on a surface loop. Attention mechanisms are well-suited to capturing these long-range dependencies.

Conserved "grammar." Evolution imposes statistical regularities on protein sequences — certain residue combinations, motifs, and covariation patterns recur because they're structurally or functionally important.

Transfer learning works. Because the "grammar" of proteins is shared across protein families, representations learned on large diverse sequence databases transfer well to downstream tasks (structure prediction, function annotation, fitness prediction, localization), mirroring how GPT/BERT embeddings transfer across NLP tasks.

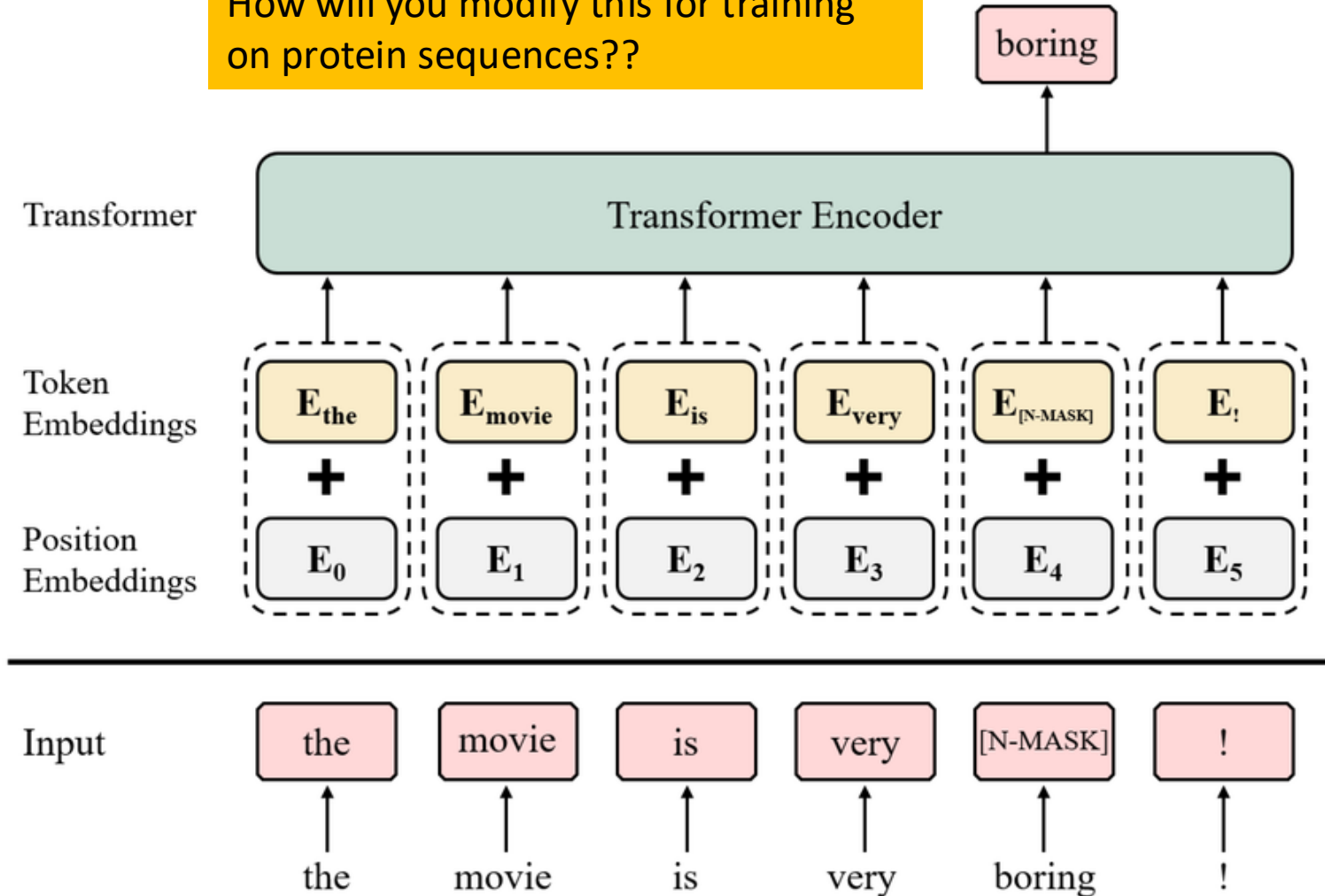
FM architecture and training for language models

Foundation models are **large** neural networks.

Trained on **large amounts** of **unlabeled** data (self-supervised learning).

Unlike more narrow traditional machine learning models, foundation models can be **easily adapted to many different tasks** without new training data.

How will you modify this for training on protein sequences??





RESEARCH ARTICLE | COMPUTER SCIENCES |



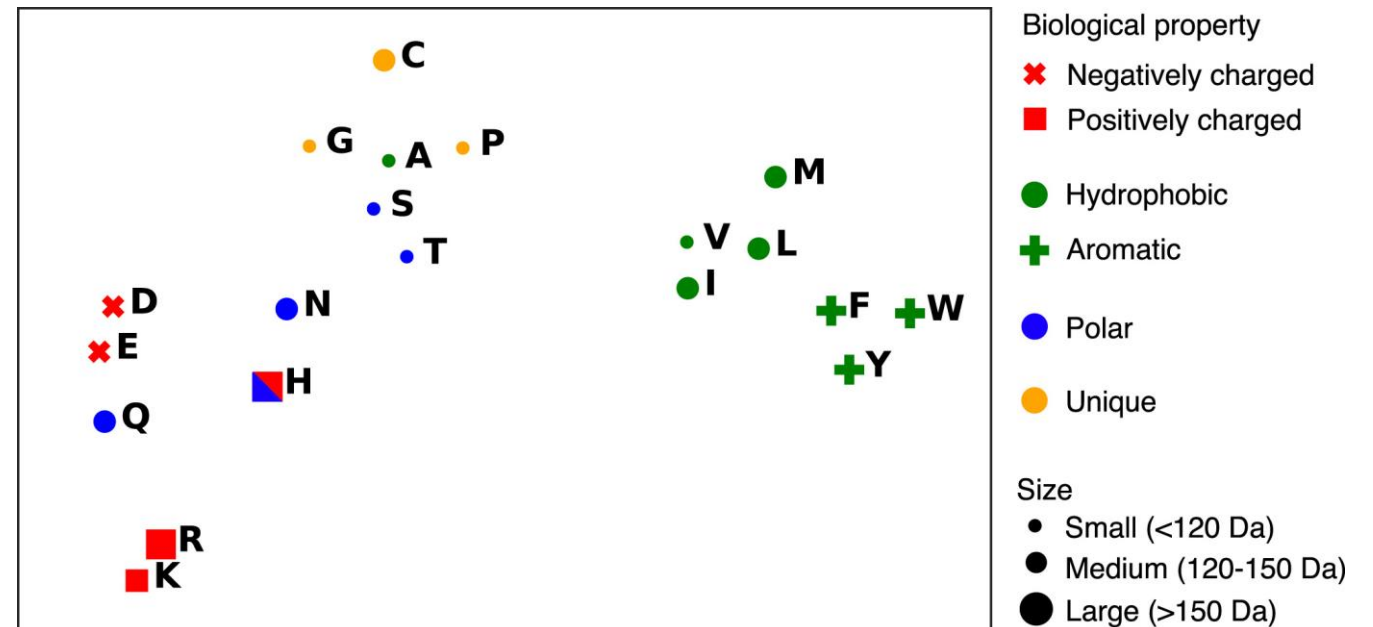
Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences

Alexander Rives , Joshua Meier, Tom Sercu , +7, and Rob Fergus [Authors Info & Affiliations](#)

Edited by David T. Jones, University College London, London, United Kingdom, and accepted by Editorial Board Member William H. Press
December 16, 2020 (received for review August 6, 2020)

April 5, 2021 | 118 (15) e2016239118 | <https://doi.org/10.1073/pnas.2016239118>

ESM
(Evolutionary
Scale
Modeling)



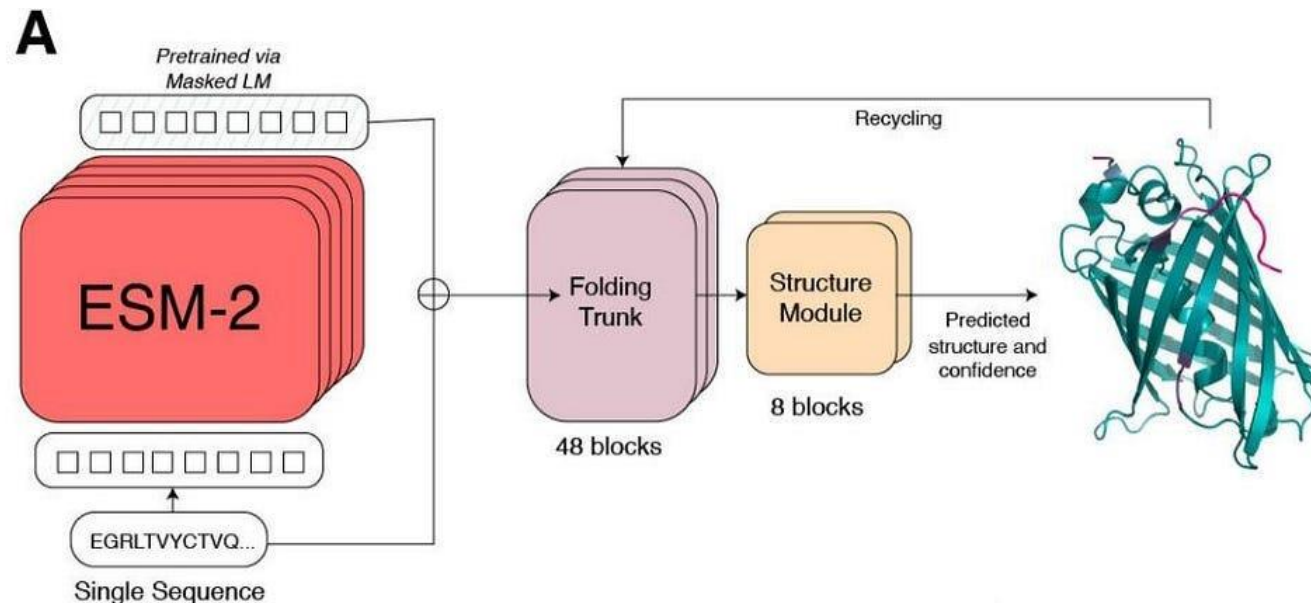
Any questions?



Evolutionary-scale prediction of atomic-level protein structure with a language model

ZEMING LIN ^{id}, HALIL AKIN ^{id}, ROSHAN RAO ^{id}, BRIAN HIE ^{id}, ZHONGKAI ZHU, WENTING LU, NIKITA SMETANIN, ROBERT VERKUIL ^{id}, ORI KABELI ^{id},

YANIV SHMUELI ^{id}, ALLAN DOS SANTOS COSTA ^{id}, MARYAM FAZEL-ZARANDI, TOM SERCU ^{id}, SALVATORE CANDIDO ^{id}, AND ALEXANDER RIVES ^{id} [fewer](#)



•**ESM-1b** (2020): The original 650M model, proved protein LMs work at scale

•**ESM-1v** (2021): Same architecture as ESM-1b but trained on UniRef90, specialized for variant effect prediction

•**ESM-2** (2022): Next generation, 8M–15B parameter range, rotary embeddings, much better per-parameter performance, enabled ESMFold for structure prediction

•**ESM-3** (2024): 98B parameters, multimodal (sequence + structure + function), generative

Hugging Face



Hugging Face (HF) is *an organization and a platform that provides machine learning models and datasets* with a focus on natural language processing.

Access the [lab notebook](#)!



Biology & Clinical Models on Hugging Face

A selection of models spanning molecular biology to clinical applications

Genomics & DNA

- DNABERT-2**
DNA language model for representation learning
- HyenaDNA**
Long-range DNA sequence modeling
- NTv2**
Modeling genetic variation effects

Proteins & Sequences

- ESM-2**
Evolutionary Scale Modeling of protein sequences
- ProtT5**
T5-based model for protein understanding
- ProteinMPNN**
Protein sequence design with GNNs

BIOLOGY MODELS

Protein Structure & 3D

- ESMFold**
High-accuracy protein structure prediction
- OmegaFold**
Efficient end-to-end protein structure prediction
- Uni-Fold**
Unified model for structure prediction

Single-cell & Transcriptomics

- scGPT**
Generative model for single-cell transcriptomics
- Geneformer**
Transformer for gene expression data
- scFoundation**
Foundation model for single-cell omics

Chemistry & Small Molecules

- ChemBERTa-2**
Pretrained model for chemical language
- MolT5**
Text-to-molecule and molecule-to-text
- GraphMVP**
Molecular property prediction with GNNs

Use cases: variant effect prediction, protein function annotation, structure prediction, drug discovery, single-cell analysis, and more.

Medical Language & EHR

- BioClinicalBERT**
Pretrained on biomedical and clinical text
- ClinicalBERT**
Language model for clinical notes
- GatorTron**
Large scale clinical language model

Radiology & Medical Imaging

- MedCLIP**
Aligning images and reports
- RadImageNet**
Image classification on radiology images
- CheXagent**
Chest X-ray analysis foundation model

CLINICAL MODELS

Pathology & Histology

- UNI**
Vision transformer for histopathology
- Prov-GigaPath**
Scaling pathology with foundation models
- H-optimus-0**
Generalist pathology foundation model

Multimodal Clinical Models

- LLaVA-Med**
Multimodal LLM for healthcare
- Med-Flamingo**
Few-shot medical multimodal model
- PMC-LLaMA**
LLM trained on PubMed Central

Other Clinical Tasks

- CT5**
Clinical trial text transformer
- MedExtract**
Information extraction from clinical text
- MedSUM**
Summarization of clinical documents

Use cases: clinical note understanding, imaging analysis, pathology, trial matching, information extraction, summarization, and decision support.

Explore these and thousands more at <https://huggingface.co/models> • Filter by task, library, or license

https://huggingface.co/docs/transformers/model_doc/esm

Transformers

Search documentation

V5.5.4

EN

159,747

DPR

ELECTRA

Encoder Decoder Models

ERNIE

Ernie4_5

Ernie4_5_MoE

ESM

EuroBERT

EXAONE-4.0

EXAONE-MoE

This model was released on 2019-04-19 and added to Hugging Face Transformers on 2022-09-30.

ESM

PyTorch

[Overview](#)

This page provides code and pre-trained weights for Transformer protein language models from Meta AI's Fundamental AI Research Team, providing the state-of-the-art ESMFold and ESM-2, and the previously released ESM-1b and ESM-1v. Transformer protein language models were introduced in the paper [Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences](#) by Alexander Rives, Joshua Meier, Tom Sercu, Siddharth Goyal, Zeming Lin, Jason Liu, Demi Guo, Myle Ott, C. Lawrence Zitnick, Jerry Ma, and Rob Fergus. The first version of this paper was [preprinted in 2019](#).

```
from transformers import AutoTokenizer, AutoModel
import torch

# Load pre-trained ESM-2 model and tokenizer
model_name = "facebook/esm2_t33_650M_UR50D" # 650M param version
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

model.eval()

# Example protein sequence
sequence = "MKTVRQERLKSIVRILERSKEPVSGAQLAEELSVSRQVIVQDIAYLRSLGYNIVATPRGYVLA"

# Tokenize and get embeddings
inputs = tokenizer(sequence, return_tensors="pt", add_special_tokens=True)
# Input: 1 sequence of 61 residues + 2 special tokens (cls + eos) = 63 tokens

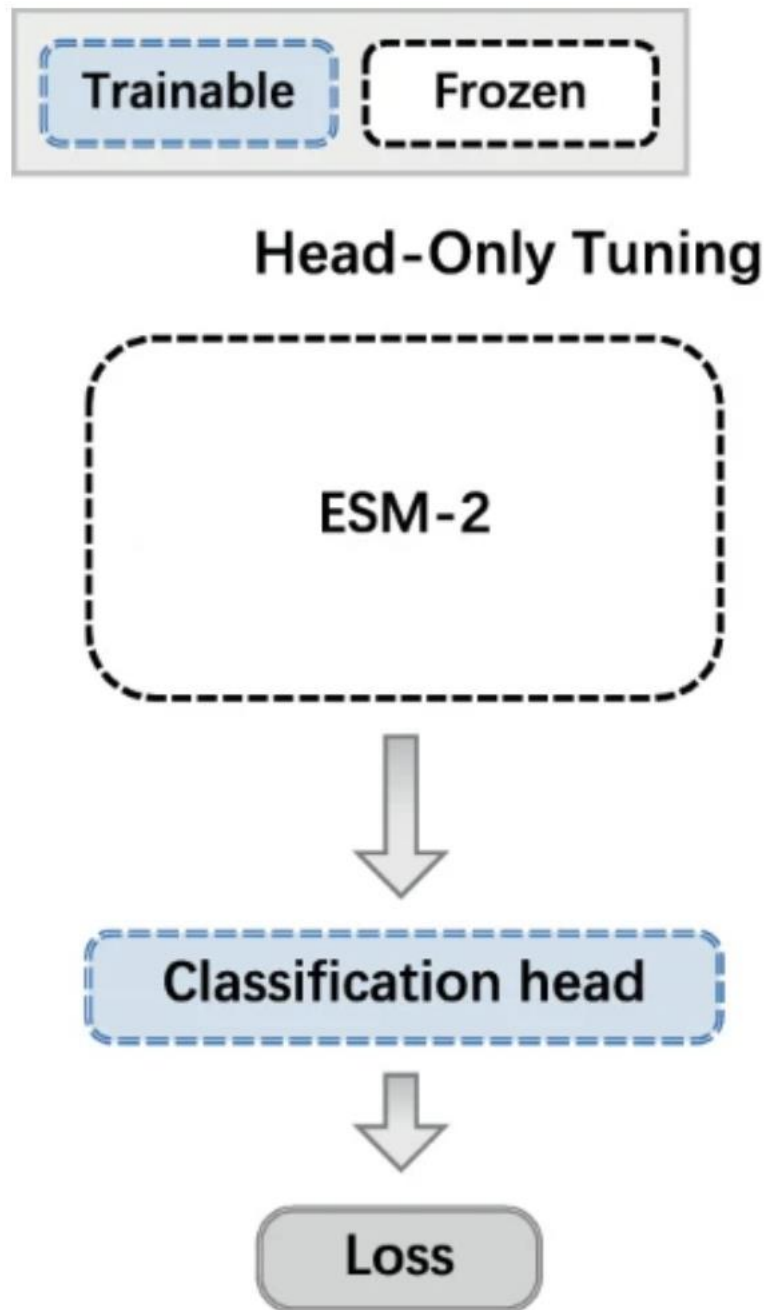
with torch.no_grad():
    outputs = model(input_ids=inputs["input_ids"], attention_mask=inputs["attention_mask"])

# Per-residue embeddings (batch, seq_len, hidden_dim)
residue_embeddings = outputs.last_hidden_state

# Mean-pool over residues for a single sequence-level embedding
# (exclude BOS/EOS tokens at positions 0 and -1)
sequence_embedding = residue_embeddings[0, 1:-1, :].mean(dim=0)

print(f"Residue embeddings shape: {residue_embeddings.shape}")
print(f"Sequence embedding shape: {sequence_embedding.shape}")
```

Suppose you had to do
function prediction,
what would you do
next?



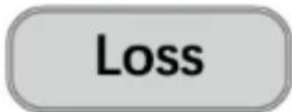
Head-only training means you freeze the pretrained ESM weights and only train a small task-specific layer on top (e.g. a linear layer or small MLP)

Head-only is preferable when:

- Have a small labeled dataset (hundreds to low thousands of examples). Updating millions of ESM parameters on little data risks overfitting, while training a small head is much safer.
- Want fast, cheap iteration. Frozen ESM means you compute embeddings once, cache them, and then training the head takes seconds on a CPU.
- **The task aligns well with what ESM already learned.** Things like secondary structure prediction or conservation scoring are "close" to what masked language modeling captures, so the frozen representations are often good enough.

Fine-tuning

Fully fine-tuning



Fine-tuning means you also update the ESM weights themselves.

Fine-tuning is preferable when:

- Have a larger labeled dataset (thousands to tens of thousands+) and can afford the compute.
- The task is "far" from pretraining. For example, predicting a specific experimental property like thermostability or binding affinity to a particular target. The pretrained representations may need adjustment to capture task-specific signal.
- Have plateaued with head-only training and need that extra performance.
- **The sequence distribution in your task is very different from the pretraining data** (e.g. synthetic or heavily engineered sequences).

In practice, the typical workflow is:

1. Start with head-only training as a baseline
2. If performance is insufficient, try fine-tuning with a low learning rate for the ESM layers (e.g. $1e-5$) and a higher rate for the head (e.g. $1e-3$)
3. **Use LoRA or similar parameter-efficient methods** as a middle ground: you update a small number of adapter parameters inside ESM without modifying the full weights, getting most of fine-tuning's benefit with less overfitting risk and lower memory cost

Any questions?



LoRA Finetuning

LoRA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS

Edward Hu*

Yelong Shen*

Phillip Wallis

Zeyuan Allen-Zhu

Yuanzhi Li

Shean Wang

Lu Wang

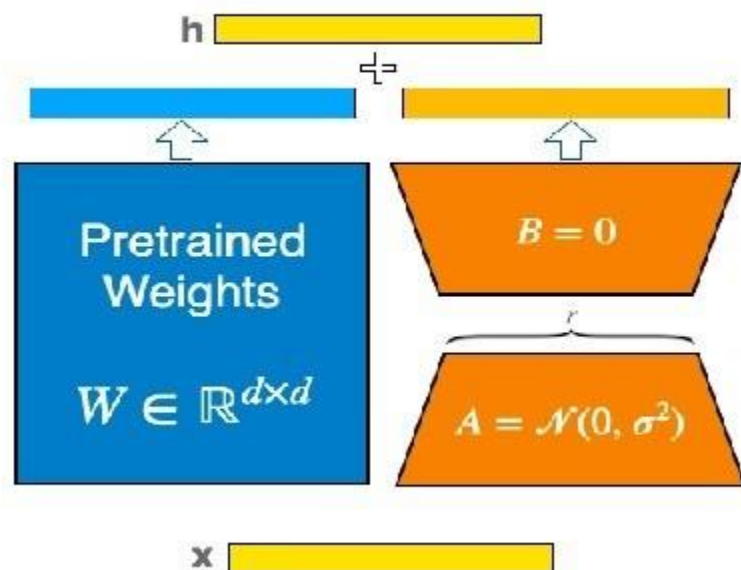
Weizhu Chen

Microsoft Corporation

{edwardhu, yeshe, phwallis, zeyuana,
yuanzhil, swang, luw, wzchen}@microsoft.com

yuanzhil@andrew.cmu.edu

During training



$$h = Wx + BAx$$

$$h = \underbrace{(W + BA)}_{W_{merged}}x$$

After training



Instead of learning a full update ΔW , LoRA factorizes it into two smaller matrices:

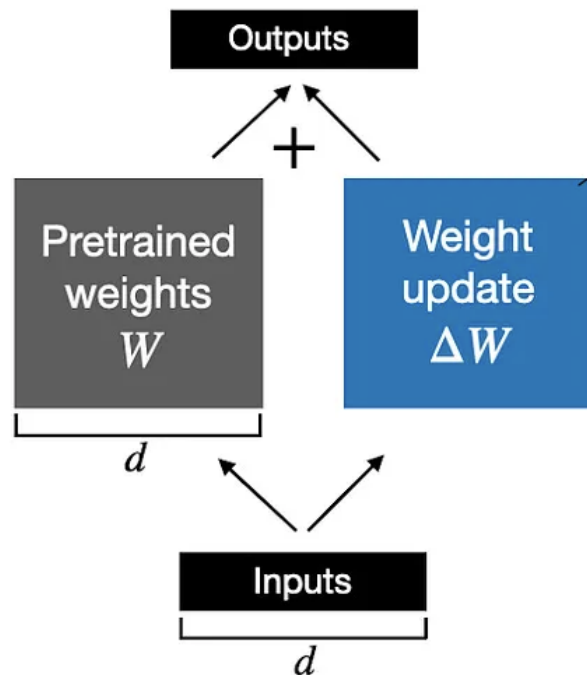
$$\Delta W \approx AB$$

where:

- A is $d \times r$
- B is $r \times k$
- $r \ll d, k$ (this is the "low rank")

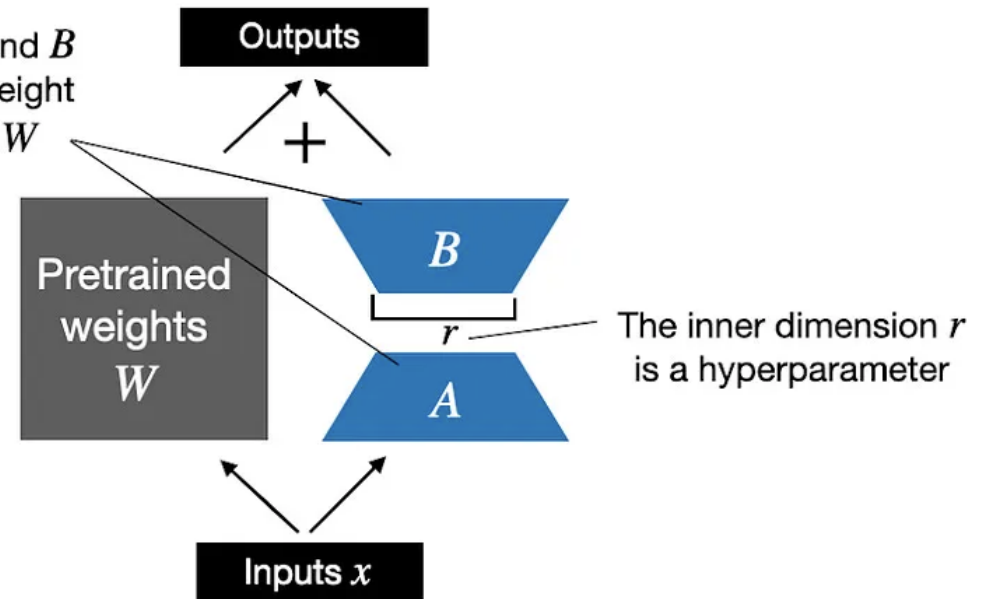
The *change* you need to make to W can be approximated by a **low-rank matrix**.

Weight update in **regular finetuning**



Weight update in **LoRA**

LoRA matrices A and B approximate the weight update matrix ΔW



```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from peft import LoraConfig, get_peft_model, TaskType
# PEFT: Parameter-Efficient Fine-Tuning
import torch

# Load ESM-2 with a classification head
model_name = "facebook/esm2_t33_650M_UR50D"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name, num_labels=2)

# Configure LoRA
lora_config = LoraConfig(
    task_type=TaskType.SEQ_CLS,
    r=8,                      # rank of the low-rank matrices
    lora_alpha=16,            # scaling factor
    lora_dropout=0.1,         # dropout on LoRA layers
    target_modules=["query", "value"], # which attention matrices to adapt
)

# Wrap the model with LoRA
model = get_peft_model(model, lora_config)

# See how few parameters you're actually training
model.print_trainable_parameters()
# e.g. "trainable params: 1.8M || all params: 652M || trainable%: 0.28%"
```

```
# Training loop (minimal example)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)
sequences = ["MKTVRQERLKSIVRI", "AACDEFGHIKLMNPQ"]
labels = torch.tensor([0, 1])
inputs = tokenizer(sequences, return_tensors="pt", padding=True,
add_special_tokens=True)
inputs["labels"] = labels

model.train()
outputs = model(
    input_ids=inputs["input_ids"],
    attention_mask=inputs["attention_mask"],
    labels=inputs["labels"],)

loss = outputs.loss
loss.backward()
optimizer.step()
optimizer.zero_grad()
print(f"Loss: {loss.item():.4f}")

# Save and reload the LoRA adapter (only saves the small adapter weights)
model.save_pretrained("esm2_lora_adapter")
# Later: model = AutoModelForSequenceClassification.from_pretrained(model_name,
num_labels=2)
#         model = PeftModel.from_pretrained(model, "esm2_lora_adapter")
```

Recap– learn about protein language models and how to use them

(1) Protein Language Models

(2) Hugging Face

(3) LoRA fine-tuning

(3) Class activity: ESM model for protein sequences

Wrap up



What was the clearest point today?

What was the muddiest point today?

